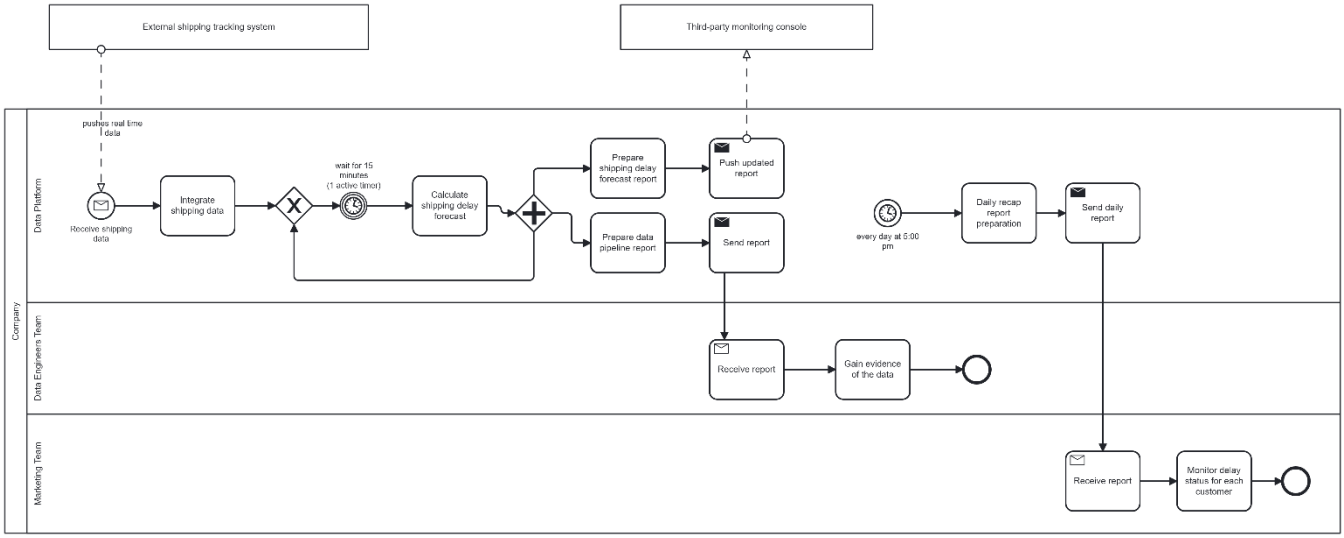
# Information Systems Design Project Work

Student: Nicola Laterza

## BPMN



# Architecture

This solution is based on **Google Cloud**.

Here are explained the components used in each layer of the data platform.

**Landing**

The objective of this layer is to function as an intermediate level between the external sources and the Data Lakehouse by storing the data coming from the external data sources without applying transformations.

- **Pub/Sub**: since we need a message queue system to handle the data stream from the external shipments tracking system, I've chosen Pub/Sub because it is a component that allows asynchronous communication between two services, the publisher and the subscriber. Publisher sends events without regards how or when these are to be processed. So it is suitable for capturing data coming from external sources in streaming mode.
    - Pub/Sub also provides automatic scaling, low-latency across multiple regions, reliable and secure message transmission. It also seamlessly works with other Google Cloud services like Dataflow. Pub/Sub can easily handle the required 300,000 events per day and it can handle various data formats.
- **Cloud Storage**: it is a binary large-object storage, low cost, durable and high available. It always encrypts data on the server side, before it is written to disk by default using HTTPS/TLS.
    - Since the requirements need to locally store the data stream for further elaboration, we can use a Cloud Storage subscription to Pub/Sub to save events as they are received. Cloud Storage as a subscription to Pub/Sub ensures exactly-once delivery semantics, which is good for maintaining data consistency.
    - *Note*: this Cloud Storage bucket is useful for storing the raw data stream without any data processing. Another option is to avoid this bucket in the landing layer and create a Dataflow subscription to Pub/Sub that performs some basic processing (such as masking personal information) and stores the data in L0 layer. I decided not to use this option because:
        - considering the requirement, I suppose to store the real raw data stream coming from the external system without elaboration
        - and the overhead of running a Dataflow job is not necessary when messages don't require additional processing before storing them in a Cloud Storage bucket
    - Of course, since we store in this bucket the data before anonymization, they are encrypted and typically not accessible by the Data Engineers team.

- We can also use this storage in an ephemeral way, deleting data after being processed by Manipulation layer.
- Storage class: Standard Storage, best for frequently accessed and/or stored for only brief periods of time.

**Data Manipulation**

This architectural layer has the task of carrying out transformations, aggregations and operations on the data stored in the Data Platform and in transit on the different levels that compose it. Specifically, it performs technical data quality and functional data quality.

- **Cloud Dataflow**: it is a fully managed service to transform and enrich data in stream (real time) and batch (historical) modes. It permits to write complex data pipeline in a serverless approach, automating infrastructure provisioning and cluster management. It is based on Apache Beam SDK open source programming model.
  - Dataflow automatically scales to handle varying loads.
- **Cloud Scheduler**: it is a fully managed, serverless job service that allows you to schedule virtually any task, including running batch jobs, big data processing, or triggering services at regular intervals.
  - In our scenario, it is useful to trigger the data elaboration pipeline every 15 minutes and to trigger the report sending to the marketing team once a day at 5:00pm.
  - An alternative to Cloud Scheduler can be Kubernetes CronJobs, but they are more complex to manage, they add overhead and they are better for advance / resource-intensive scheduling

**Data Lake**

The Data Lakehouse architectural layer represents the services that have the objective of persistently storing the data involved in the processes of the Data Platform. At this stage, data is managed in different layers: L0, L1, L2.

- **Cloud Storage** (**L0** raw storage): component already described above
  - a Storage Bucket is not particularly high-performance, but economical, scalable and which can even allow the storage of files in compressed format. Data arrives as input from the Data Landing Layer and is transformed through the Data Manipulation Layer
  - since the provided dataset has email and password anonymized, I suppose that we should do at least some light processing such as masking personal information, so I decided to store anonymized data in L0 raw storage, after the basic processing done by Dataflow.

    - We can implement lifecycle policies in Cloud Storage to automatically transition or delete older data (completed shippings) for cost optimization.
- **BigQuery** (L1 curated and L2 ready): it is a cloud-native, fully managed enterprise data warehouse supporting large-scale analytics. It offers high-performance analysis of large data sets, with automatic scaling up and down to maximize query performance and cost. BigQuery also decouples compute and storage to provide cost-effective resources that are unavailable to on-prem users or to cloud data warehouses based on legacy technology. It is optimized for querying structured, processed data and it's efficient for large amount of data
    - The analytical database is a good solution for storing refined data and aggregated/enriched data. This enables aggregate data to be represented by similar characteristics, in datasets on which to implement querying and analysis logic
    - We can use BigQuery's partitioning and clustering features to optimize query performance and reduce costs.
    - We should implement an appropriate access controls on both Cloud Storage and BigQuery to ensure data security and compliance.

**Consumption**

The data consumption layer is where data is retrieved from Data Lakehouse L2 layer to be used for elaboration. Here are implemented business use cases through custom application components, advanced analytics algorithms and AI models.

- **Google Kubernetes Engine** (GKE): it provides a managed environment for deploying, managing, and scaling containerized applications using Kubernetes, an open-source container management system. It consists of multiple machines grouped together to form a cluster.
    - The engine abstracts the complexities of managing Kubernetes clusters, providing users with automated operations, seamless integration with the cloud ecosystems, and scalability to accommodate varying workloads. It also has features such as monitoring, logging, and security controls.
    - I decided to use GKE because of its flexibility, it is open to custom implementation, for example we can run our algorithms using our preferrred programming language. We can also implement custom components to push data on external data storage or implement interfaces to manually start the data elaboration process and so on.
    - In our scenario the algorithm is implemented using PySpark, we can run a **Spark** application on a Kubernetes cluster.
    - An alternative component to GKE that we could use is **Dataproc**, that is a fully managed and scalable service provided by Google Cloud for running Apache

Hadoop, Apache Spark, and other open-source data analytics frameworks. Instead, Vertex AI can be an overkill solution in this case.
    - Note: I assume that the training phase has already been done, because in the brief there is only the need of updating the shipment delay forecast value every 15 minutes. The application reads the model file saved after the training phase and set it on inference on the new data.

**Exposure**

The data exposure layer implements capabilities to expose data to third parties in a standard and secure way.

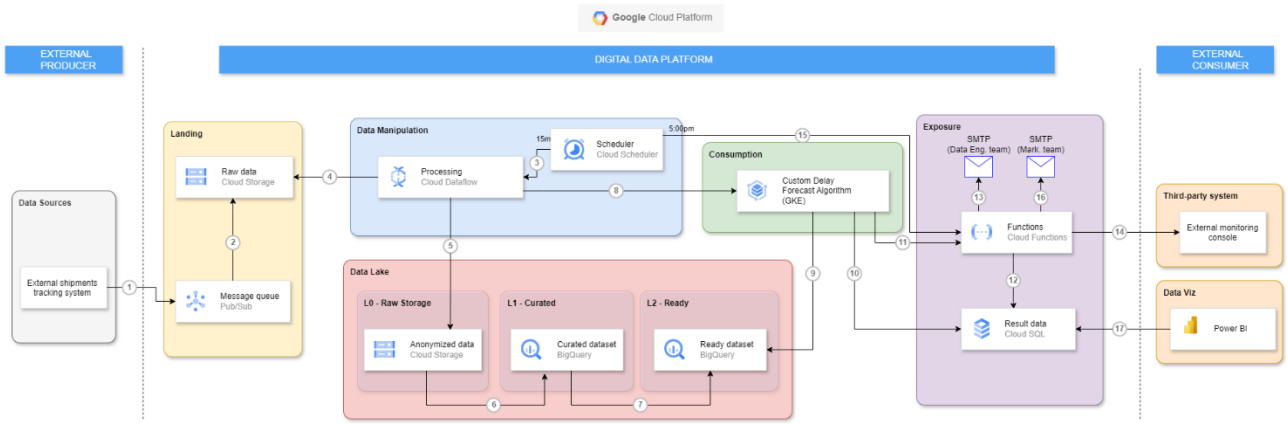
- **Cloud Functions**: it is a lightweight serverless compute solution for developers to create single-purpose, stand alone function that can be trigger by cloud event. It scales automatically in response to the events. It is optimal for asynchronous workloads like lightweight ETL and cloud automation API.
    - I use it to send the different reports
- **Cloud SQL**: it provides a cloud-based fully managed alternative to local MySQL, PostgreSQL, and SQL Server databases. It allows the user to manage the data, while it manages backubs, high availability and failover, maintenance and updates, monitoring, and logging. It's a cheaper alternative than an analytic engine.
    - It ensures transactional integrity, but if we would need to store large-scale forecast results, we can consider to use BigQuery.
    - Note: this storage is not striclty necessary, because we could eventually push the results to the external system without storing the results, but I think that storing the results is better to facilitate exposure and reporting, for auditing and compliance, historical analysis and fault tolerance.
    - Cloud SQL is a good solution to implement Power BI dashboards and let data be accessed by other components to send data to third-party systems.

Notes about components **location**:

- We can use a Region or Multi-region location for our data platform components. This choice depends on the requirements and trade-off between performance, cost efficiency, disaster recovery, compliance with data privacy regulations, etc.
- Since there are not explicit requirements of this type, for the sake of simplicity, I will consider a **Region** location (in particular the datacenter in Milan: europe-west8) to prioritize performance (low-latency access), cost-efficiency (lower costs), data privacy and simplicity.
- Of course, there are some disadvantages: users or systems in other parts of the world may experience higher latency; a single-region is more vulnerable to region-wide

outages compared to a multi-regional setup; there may be limitations on scaling beyond the capacity of a single region; etc.

## Architecture diagram




## Data flow

1. An external shipping tracking system collects events data from all shipments and pushes data in real time to the data platform. This data stream is received by Google Pub/Sub component. The external system acts as the publisher that creates and sends event messages to a topic. One or more subscribers can create a subscription to the topic to receive the messages from it. The event messages contain the new shipping data (customer, product, order total, etc.) and updated status in a predefined data structure.
2. The raw data stream is locally stored in a Cloud Storage bucket for further elaboration. Data is not processed and the data flow is fast.
3. Every 15 minutes, Cloud Scheduler triggers a Dataflow job for starting the data elaboration pipeline.
4. Cloud Dataflow retrieves data from Cloud Storage and performs some light processing, specifically it anonymizes customer personal information (email and password).
    a. It can also perform basic technical data quality, such as checking the presence of data and that it respects standards (missing data, wrong format, etc.)
    b. For Dataflow we can consider using incremental processing. Dataflow allows for windowing and event time processing to handle only the latest updated data

(last 15 minutes) rather than reprocessing all historical data. This would reduce computational cost and latency.

5. The anonymized data is stored in L0 raw storage layer.
6. Then, Dataflow performs other data refinements and functional data quality (checks if data makes sense from a business point of view). The resulting curated data is stored in BigQuery (L1).
    a. In particular, it controls if data is received aligned with business rules (i.e. order date is lower than shipping date; numbers within some ranges; categorical features contains the predefined values, etc.)
7. Subsequently, data is grouped, deduplicated and prepared to be consumed by the shipment delay forecast algorithm. The resulting ready data is stored in BigQuery (L2). For example:
    a. we select only the data collected during the past hours (as indicated in the process description)
    b. We select/extract only the relevant features used by the algorithm
    c. We compute the weight column for each shipping
8. After data has been completely elaborated in the data lake, the process triggers the execution of the forecast algorithm in GKE.
9. A Spark Application is executed on a Kubernetesc Cluster. In particular, the algorithm in the GKE reads the ready data from Big Query in L2 layer, reads the model file saved after the training phase and set it on inference on the new data.
10. So, it recalculates the shipping delay forecast based on the data collected during the past hours. The elaboration results and data are stored in a relational database (Cloud SQL) to facilitate reporting and exposure
11. Once the algorithm stores the results in Cloud SQL, it automatically triggers a Cloud Function
12. This Cloud Function retrieves data from Cloud SQL and creates an updated Shipping Delay forecast report and a data pipeline report.
13. The Function sends the data pipeline report to the Data Engineers team via email (using SMTP)
14. The Function pushes the Shipping Delay forecast report to an external third-party monitoring console by means of an API call.
15. Once a day at 5.00pm, a Cloud Scheduler job triggers a Cloud Function to create a daily Shipping Delay forecast recap report
16. Then, the Function sends the report via email to the marketing team for monitoring and control the status of shipments delay for each customer.
17. The elaboration results stored in Cloud SQL can be analyzed by Power BI tool to create an updated dashboard for a better understanding of the forecast values.
    a. Power BI component is not explicitly required in the process description but, since the project work involves also the creation of a Power BI dashboard, I consider it in the data platform.

# Costs estimation

The following table shows the running costs estimation (per month) of my solution.

Requirements:

- Average number of events collected per day: 300.000
- Average size for each event message: 5KB
- Average size per month = 45GB

Region: Milan (europe-west8)

| Item | Price driver | Reference | Usage amount per month | Unit price $ | Total price per month |
|---|---|---|---|---|---|
| Pub/Sub (Basic, 1 subscription) | Volume of published/read data (GB) | https://cloud.g | 45 | 0,062 | 2,78 |
| Cloud Storage [Landing] (Standard storage) | Volume (GB) | https://cloud.g | 45 | 0,023 | 1,04 |
| Cloud Dataflow (Classic, General purpose n1-standard-2 workers in Batch Mode) | Usage time (h) | https://cloud.g | 730 | 0,147 | 107,61 |
| Cloud Scheduler (2 jobs) | N. jobs (first 3 jobs free) | https://cloud.g | | | 0,00 |
| Cloud Storage [Lake L0] (Standard storage) | Volume (GB) | https://cloud.g | 45 | 0,023 | 1,04 |
| BigQuery [Lake L1, L2] | Volume of query data (TB, first 1TB free) | https://cloud.g | 5 | 7,810 | 31,24 |
| GKE standard edition (n2-standard-2, 3 node pool) | Usage time (h) | https://cloud.g | 730 | 0,439 | 320,41 |
| Cloud SQL - instance db-standard-2 (2vCPUs, 7.5GB RAM) | Usage time (h) | https://cloud.g | 730 | 0,157 | 114,28 |
| Cloud SQL - storage | Volume (GB) | https://cloud.g | 30 | 0,197 | 5,91 |
| Cloud Functions (1st gen) | (2 milion invocations per month in free tier) | https://cloud.g | | | 0,00 |
| | | | | | |
| | | | | **TOT ($/month):** | **584,30** |

Notes:

- To compute the costs estimation, I used the pricing web pages (reference) and Google Cloud's pricing calculator
- The price and usage amount are approximate values because I have not considered the specific details of the single components
- The components dimensioning is not necessary the optimal one